Homework #1

This HW assignment has two parts.

Part 1 (problems listed on this page) pertains to the photoelectric effect, the material covered in Thursday's class; the problem statements can be found in "Problems from Chapter 3.pdf" on the course website. Please upload a write-up of your solution to Part 1 as a single PDF file.
Name the file "**Lastname_HW1_Part1.pdf**"

Part 2 focuses on some basic proficiency in MATLAB. I recommend doing part 2 (pages 2-7 of the assignment) first. Upload part 2 output – code listing(s) and output(s) as single PDF file; the easiest way to do so is to use MATLAB's publish feature.
Name the file "**Lastname_HW1_Part2.pdf**"

Part 2 will also require you to create a movie. This will be your third upload.
Name the avi file "**Lastname_HW1_Movie.avi**"

>>Therefore, you will upload a total of 3 files. <<

Part 1 problems (from Chapter 3, see scanned pages):

<u>The photoelectric effect</u>

3-24

3-26

3-27

3-28

3-29

3-31

3-32

3-42

3-45

HW 1, Part 2: Introduction to MATLAB

Part 1: The Basics

MATLAB lets you perform numerical computations and make figures of your results. It's available in all the CAE machines, both in Windows (under CAE Applications) and Unix (type "matlab" at the command prompt). MATLAB is also available through the library as tethered software, if you want to install it on your personal computer. This lab will assume you're using the Windows version.

NOTE: MATLAB is a very popular tool and has been developed, over many releases, to cater to the scientists that use it. The plus side of this is that whatever you need to do, there's probably a command for it. The downside is that there's a HUGE number of available commands. To ever be good at MATLAB you need to get comfortable finding useful commands and learning to use them from the help documentation. We have two goals in this lab: 1) to teach you the basic structure of how you'll use MATLAB and a few necessary commands you'll need, and 2) to show you how to find more information when you need it.

You'll turn in:

- A printout of your final m-file showing your code for all calculations and figures
- A lab write-up. The write-up may be typed or legibly handwritten. It should include
    - solutions to all underlined questions in the lab
    - all relevant plots and figures
    - any short-answer descriptions that are asked for.

1) Using M-files

You can perform computations by entering the commands for them, using the correct syntax, at the command prompt ">>". The command prompt is a good place to test out your syntax or to run quick throw-away computations ("what's the square root of 2, again?") but you don't want to rely on it too much, because once you close MATLAB (or if MATLAB crashes while you're using it) everything you were doing in the command prompt is gone.

To keep from losing your work, you want to be putting most of your work into m-files, which end with the file extension ".m". Open a new m-file (go to File→New→M-file) and save it with whatever name you'd like (for example, "Lab1.m" without the quotes). Any command you put into this m-file will run exactly the same if you type it into the command prompt, but here you can keep track of it easily, and you can check over what you've done to make sure it's right.

Plenty of what used to be doable from the command line can be done through appropriate menus in newer versions of MATLAB

All MATLAB calculations and plots in this lab should be entered into your m-file (except where explicitly stated otherwise.) Don't erase chunks of your m-file when you've answered the questions, because you need to turn it in with your answers.

2) Defining arrays

Data is input into MATLAB two ways: you can type it into an array manually, or you can have MATLAB read it in from a file. We'll start with putting data in manually.

All numbers in MATLAB go into arrays, and the arrays can be practically any size and have any number of dimensions. The values held in the array are its elements. If you say

```
var_1 = 17
```

you're defining an array called `var_1` which has one row and one column, and giving its single element the `value 17`. You can define a 1D array with incrementally growing element values by typing

```
var_2 = 4:10.
```

Type this into the command prompt. <u>What does MATLAB return? Explain what the colon ":" is doing in a sentence or two. Then try these,</u>

```
var_2 = 4:2:10
var_2 = 10:-3:4
var_2 = 4:4:10
```

<u>Show the output and explain what's happening for each of these. Are the bounding numbers included in each version of var_2?</u> For more information, use `help colon`.

To define a larger array given by

$$\texttt{mat_1} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

in MATLAB, you'd type

```
mat_1 = [1 2 3 ; 4 5 6]
```

Define the following arrays in your m-file.

$$\texttt{a} = \begin{bmatrix} 1 & 3 & 7 \\ 2 & 5 & 6 \end{bmatrix}$$

$$\texttt{b} = \begin{bmatrix} 4 & 7 & 9 \\ 8 & 1 & 3 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\texttt{c} = \begin{bmatrix} 1-j & 1+j & 3 \\ 2-4j & 8+3j & 6j \\ 0 & 0 & 7 \end{bmatrix}$$

The imaginary number $j = \sqrt{-1}$ is recognized by MATLAB (unless you accidentally define *j* to have some other value, which you should avoid doing.) To make sure you've entered your arrays correctly, save the m-file and run it. You can run m-files two different ways, either by typing its name without the file extension, `Lab1`, into the command prompt, or by clicking the "Run" button at the top of the Editor window.

If you've done all this correctly so far, the arrays you've entered will be shown in the command window. You don't want every single command to be shown executing in the command window, so once you've checked your arrays go put semicolons ";" at the ends of the array-definition lines in your m-file (a semicolon at the end of a line will suppress output to the command window).

You can access elements of arrays using indices. For example, the element in the second row and third column of matrix `c` is `c(2,3) = 6j`. In general, `c(m,n)` will give you the element in the mth row and n^{th} column of `c`. You can access several elements with vectors instead of single numbers. <u>In your write-up, compare `b` and `b(1,1:2)`, and explain what MATLAB returns for each when you type them in the command window. Also explain the output for `b(1:3,2:3)` and `b(:,1:3)`.</u>

3) Manipulating Arrays as Matrices
MATLAB's extensive help files are all accessible through the command prompt. Type

```
help arith
help inv
help transpose
```

or for more info try typing `helpwin`. In your m-file, compute the quantities given below using the help files to find the right commands. <u>Include the results for each computation in your write-up.</u> Note that you should be doing matrix multiplication, not array (element-by-element) multiplication.

i) bc
ii) $b^{-1}c$
iii) ab
iv) b + c
v) ba^T
vi) ab(1:3,1:2)

The last of these multiplies `a` by a region of `b`. Most commonly-used commands are behind the links in `helpwin`. Go look in `matlab\elfun`, for instance, and `matlab\lang`. You'll need a lot of these commands eventually.

Part 2: File I/O

MATLAB allows you to save data in files, and read it in later. The simplest way to do this is using the commands `save` and `load`. Typing in `save A` saves matrix `A` to a file called `A.mat`. If you want to read in matrix `A` later, just type `load A`.

You can also use the load command to read in ASCII files, as long as they are formatted correctly, so that the number of columns in each line is the same and the columns are delimited with a space. Suppose you have a file called `datafile.dat` that contains the following lines:

```
12.5 6 9
```

```
1 3.5 125
2 4 0
```

Notice that the columns do not have to be lined up exactly, as long as there are the same number of them on each line. You can read this into your MATLAB m-file by typing load `datafile.dat` in your m-file. MATLAB will read the file contents into a matrix called `datafile` (without the file extension).

For reading and writing binary data or poorly formatted ASCII files, the following commands are used: `fopen`, `fread`, `fscanf`, `fwrite`, `fprintf`, and `fclose`.

There's a file in the course website called Lab1_data.mat. Download it and put it in the same folder as your working m-file. In your m-file, after the work you've already done, <u>load the downloaded file into MATLAB, and use some "general purpose" and "elementary matrix" commands to</u>

1. <u>List the names of the variables in the file</u>
2. <u>Give the array sizes of the variables</u>

<u>List which commands you used.</u>

Part 3: Control statements

Look up `linspace`. In your m-file, use `linspace` to define a row vector named `xx` that has 400 equally spaced points that vary in value from -5 to 5. Now calculate a function `fx1`.

```
fx1=sin(3*pi*xx).*exp(-xx.^2);
```

This defines `fx1` in terms of `xx`., using array multiplication. Now define another function, `fx2`, that's the same as `fx1` except that the magnitude of `fx2` cannot exceed 0.5.

```
fx2=zeros(size(xx));
for ii=1:length(xx)
     if abs(fx1(ii))<0.5
          fx2(ii)=fx1(ii);
     elseif fx1(ii)>0.5
          fx2(ii)=0.5;
     elseif fx1(ii)<0.5
          fx2(ii)=-0.5;
     end
end
```

<u>Look up the command `while`, and use a `while` loop to find the first value of `xx` where `fx1` is greater than `fx2`. Include this in your m-file and put the answer in your write-up</u>.

Part 4: Figures and movies

Now try plotting your functions.

```
plot(xx,fx1)
```

```
xlabel('x')
ylabel('f(x)')
```

You can change the line color, line thickness, font size, and so on. MATLAB makes publication-worthy figures, though it can take some tweaking. Type `help plot` into the command window and make a figure that has `fx1` and `fx2` plotted on the same set of axes. Label your plots with a legend:

```
legend('fx1','fx2')
```

To save a jpeg of your figure, type

```
print('-djpeg', 'your_file_name.jpg')
```

Turn this figure in with your write-up.

Now look at the data you loaded into MATLAB before. The 2D array named `f0` is concerned with electron velocity in a doped semiconductor (a semiconductor impregnated with electrons that are not bound to a particular atom). The unbound electrons in a doped semiconductor have velocity corresponding to their energy, which is determined, in this case, by the temperature of the medium. The array `f0` is defined so that each element corresponds to the probability of finding an electron with a particular velocity. The velocity vectors are included as `vx` and `vy`, with units of m/s.

Make three separate images, using `imagesc`, `waterfall`, and `surf`. Label all axes with units (except `f0`, which is a probability and so is unitless). Include color information in each figure with the command `colorbar`. Save each figure with a distinct filename, and turn the figures in with your write-up.

Making a movie: This example will lead you through making a simple movie in MATLAB of 3D data. Type the following into your m-file.

```
for j=1:100
    plot3(xpos(j,:), ypos(j,:), zpos(j,:),'r.');
    grid on;
    pause(0.1);
end
```

The pause(0.1) is necessary to see the images; otherwise the computations get ahead of the image rendering and you don't see anything till it's done. You can label your axes and define the axis range, by putting

```
axis([0 4e-7 0 4e-7 0 1e-7])
xlabel('X-axis (m)');
ylabel('Y-axis (m)');
zlabel('Z-axis (m)');
```

inside the `for` loop, before the `pause` statement. Look up the commands `getframe` and ~~`movie2avi`~~, and use them to produce an avi file containing your movie of electrons moving and scattering within dielectric. Later in the semester as you're asked to make movies in MATLAB, this is the method you will use. Upload your movie file to the course website.

VideoWriter

More on movies on the next page!

Making a movie using VideoWriter. Below is the typical syntax:

```
movObj=VideoWriter('MovieName.avi')
movObj.FrameRate = 10; %can change frame rate, obviously
open(movObj)

figure(1)

for j=1:NumberOfSteps
%whatver you need to plot a single frame
plot(x,y,'b-') %just an example
xlabel('xlabel')
ylabel('ylabel')
%other attributes, like fixing axis range axis ([0 1 0 2])
writeVideo(movObj,getframe(gcf)); % look up what gcf means!
end
close(movObj)
```

<< Don't forget to close movObj!